



**BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY**
(Empowered Autonomous Institute Affiliated to University of Mumbai)
[Knowledge is Nectar]

Department of Computer Science Engineering

<MAKE A COPY OF THIS TEMPLATE FOR EACH LAB>

Course – Software Engineering Lab (SE)

UID	2024800038
Name	Atharva Pramod Hule
Class and Batch	TY CSE Batch A3
Date	24/08/2026
Lab #	4
Aim	Design UML diagram -Interaction diagrams
Objective	1. Identify actors, objects, and system components involved in a use case. 2. Represent the chronological flow of messages among objects. 3. Distinguish synchronous calls, return messages, and conditional interactions. 4. Develop a UML sequence diagram for a selected system scenario. 5. Understand how interaction diagrams support object-oriented analysis and design.
Theory	A UML Interaction Diagram is a behavioral UML diagram used to represent how different actors and system components communicate with each other while carrying out a particular use case. It mainly focuses on the messages exchanged between participants and the order in which those messages occur. For the Green Finance system, the interaction diagram represents the flow of a carbon-credit transfer, beginning with the user request and continuing through validation, credit transfer, transaction creation, ledger recording, and success or failure notification. Interaction diagrams are commonly used to represent the dynamic behavior of a system. The major types are: 1. Sequence Diagram – Represents the exchange of messages between participants in chronological order, from top to bottom. 2. Communication Diagram – Focuses on the links between objects and the messages exchanged through those links. 3. Interaction Overview Diagram – Provides a high-level representation of the flow between different interactions. 4. Timing Diagram – Represents how the state or condition of objects changes with respect to time.



**BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY**
(Empowered Autonomous Institute Affiliated to University of Mumbai)
[Knowledge is Nectar]

Department of Computer Science Engineering

Elements of a UML Sequence Diagram

1. Actor – An external user, device, or system that initiates or participates in an interaction. In our Green Finance system, the User is the actor who initiates the carbon-credit transfer.
2. Participant/Object – A system component or object involved in the interaction. In our diagram, the main participants are Green Finance API, Carbon Finance Service, CarbonCredit, Transaction, and LedgerEntry.
3. Lifeline – A vertical dashed line associated with a participant. It represents the participant's existence and participation during the interaction.
4. Activation Bar – A narrow vertical rectangle placed on a lifeline to indicate the period during which a participant is actively processing an operation.
5. Synchronous Message – A solid arrow with a filled arrowhead. It represents a call in which the sender waits for the receiver to complete the requested operation. For example, the Green Finance API sends processTransfer() to the Carbon Finance Service.
6. Asynchronous Message – A solid arrow with an open arrowhead. It represents a message where the sender does not wait for the receiver to finish processing. This type of message is not used in our current Green Finance interaction.
7. Return Message – A dashed arrow used to represent the return of a result, control, or acknowledgement. For example, availableAmount is returned by CarbonCredit to the Carbon Finance Service.
8. Combined Fragment – A rectangular frame used to represent control logic within an interaction, such as alternative, optional, or repeated behavior. Our diagram uses an alt fragment.

Combined Fragments

1. alt – Represents alternative execution paths, similar to an if-else condition. In our diagram, it separates the sufficient-credit and insufficient-credit scenarios.
2. opt – Represents an optional interaction that occurs only when a specified condition is satisfied.
3. loop – Represents an interaction that is executed repeatedly while a specified condition is satisfied.
4. par – Represents multiple interactions that can execute in parallel.
5. break – Terminates the remaining interaction when a specified condition becomes true.



BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY
(Empowered Autonomous Institute Affiliated to University of Mumbai)
[Knowledge is Nectar]

Department of Computer Science Engineering

Application in the Green Finance System

In our sequence diagram, the User first sends a carbon-credit transfer request to the Green Finance API. The API forwards the request to the Carbon Finance Service, which checks the available amount of CarbonCredit.

If sufficient credit is available, the Carbon Finance Service transfers the credit, creates a Transaction, and the Transaction creates both debit and credit LedgerEntry records. A confirmation is then returned to the User.

If sufficient credit is not available, the transfer is rejected and an error message is displayed to the User.

The Transaction and LedgerEntry interaction follows the Green Finance class model, where each transaction requires at least a debit and a credit ledger entry.

Example: Interaction Diagram for the Carbon Credit Transfer Process in Green Finance Platform

A user initiates a request to transfer a specified amount of carbon credit to another user. The system forwards the request to the Carbon Finance Service, which checks the availability of the requested carbon credit. If sufficient credit is available, the system transfers the carbon credit, creates a transaction, and records the corresponding debit and credit ledger entries. The system then sends a transfer confirmation to the user. If the available carbon credit is insufficient, the system rejects the transfer and displays an appropriate error message.

The identified participants and their details are as follows:

Participant	Type	Responsibility
User	Actor	Initiates the carbon-credit transfer and receives the result of the operation.
Green Finance API	Boundary Object	Receives the transfer request from the user and forwards it to the business service.
Carbon Finance Service	Control Object	Coordinates the transfer process, checks credit availability, and controls transaction creation.
CarbonCredit	Entity/Data Object	Stores carbon-credit information and provides the available credit amount and



BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY
(Empowered Autonomous Institute Affiliated to University of Mumbai)
[Knowledge is Nectar]

Department of Computer Science Engineering

	transfer operation. <table><tr><td>Transaction</td><td>Entity/Data Object</td><td>Represents the carbon-credit transfer transaction.</td></tr><tr><td>LedgerEntry</td><td>Entity/Data Object</td><td>Records the debit and credit entries associated with the transaction.</td></tr></table>	Transaction	Entity/Data Object	Represents the carbon-credit transfer transaction.	LedgerEntry	Entity/Data Object	Records the debit and credit entries associated with the transaction.
Transaction	Entity/Data Object	Represents the carbon-credit transfer transaction.					
LedgerEntry	Entity/Data Object	Records the debit and credit entries associated with the transaction.					
	The participant selection is based on the Green Finance project model, whose core domain includes CarbonCredit, Transaction, and LedgerEntry, while the project architecture follows an API → Service → Model → Database structure. The interaction follows these steps: 1. The User initiates a carbon-credit transfer request through the Green Finance API. 2. The Green Finance API forwards the transfer request to the Carbon Finance Service. 3. The Carbon Finance Service requests the available carbon-credit amount from CarbonCredit. 4. CarbonCredit returns the available credit amount to the Carbon Finance Service. 5. If the available credit is sufficient, the Carbon Finance Service requests CarbonCredit to transfer the specified amount to the receiver. 6. CarbonCredit confirms that the transfer has been completed successfully. 7. The Carbon Finance Service creates a Transaction for the carbon-credit transfer. 8. The Transaction creates a debit LedgerEntry for the sender. 9. The Transaction creates a credit LedgerEntry for the receiver. 10.The Transaction returns confirmation that the transaction has been created. 11.The Carbon Finance Service sends a transfer confirmation to the Green Finance API, which displays the successful confirmation to the User. 12.If the available credit is insufficient, the Carbon Finance Service sends a transfer-failed response to the Green Finance API, which displays an error message to the User. The debit and credit ledger-entry steps are consistent with the project model, which specifies that each transaction creates at least a debit and a credit ledger entry.						



**BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY**
(Empowered Autonomous Institute Affiliated to University of Mumbai)
[Knowledge is Nectar]

Department of Computer Science Engineering

Message Table				
No.	Sender	Receiver	Message	Purpose
1	User	Green Finance API	TransferCredit(creditId, transfer.receiverId, amount)	Initiates the carbon-credit transfer.
2	Green Finance API	Carbon Finance Service	processTransfer(userId, creditId, receiverId, amount)	Starts the transfer-processing workflow.
3	Carbon Finance Service	CarbonCredit	getAvailableAmount()	Checks the amount of carbon credit available for transfer.
4	CarbonCredit	Carbon Finance Service	availableAmount	Returns the available carbon-credit amount.
5	Carbon Finance Service	CarbonCredit	transfer(receiverId, amount)	Transfers the specified carbon credit when sufficient credit is available.
6	CarbonCredit	Carbon Finance Service	transferSuccessful()	Confirms successful completion of the credit transfer.
7	Carbon Finance Service	Transaction	createTransaction(...)	Creates a transaction for the carbon-credit transfer.
8	Transaction	LedgerEntry	createDebitEntry(...)	Records the debit entry for the sender.
9	LedgerEntry	Transaction	debitRecorded	Confirms that the debit entry has been recorded.
10	Transaction	LedgerEntry	createCreditEntry(...)	Records the credit entry for the receiver.
11	LedgerEntry	Transaction	creditRecorded	Confirms that the credit entry has been recorded.
12	Transaction	Carbon Finance Service	transactionCreated	Confirms that the transaction has been created.
13	Carbon Finance Service	Green Finance API	transferConfirmation()	Sends confirmation of a successful transfer.
14	Green Finance API	User	displayConfirmation()	Displays successful transfer information to the user.
15	Carbon Finance Service	Green Finance API	transferFailed()	Reports that the transfer cannot be completed due to insufficient credit.
16	Green Finance API	User	displayError()	Displays the transfer failure message to the user.



BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY
(Empowered Autonomous Institute Affiliated to University of Mumbai)
[Knowledge is Nectar]

Department of Computer Science Engineering

Implementation / Code	<pre>@startuml hide footbox title Green Finance - Carbon Credit Transfer Sequence Diagram actor User participant "Green Finance API" as API participant "Carbon Finance Service" as Service participant "CarbonCredit" as CC participant "Transaction" as Tx participant "LedgerEntry" as Ledger ' 1. User initiates carbon-credit transfer User -> API : transferCredit(creditId, receiverId, amount) activate API ' 2. API forwards request to service API -> Service : processTransfer(userId, creditId, receiverId, amount) activate Service ' 3. Check available carbon credits Service -> CC : getAvailableAmount() activate CC CC --> Service : availableAmount deactivate CC alt Sufficient credit ' 4. Transfer carbon credit Service -> CC : transfer(receiverId, amount) activate CC CC --> Service : transferSuccessful() deactivate CC ' 5. Create transaction Service -> Tx : createTransaction(...) activate Tx ' 6. Create debit ledger entry Tx -> Ledger : createDebitEntry(...) activate Ledger</pre>
------------------------------	--



**BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY**
(Empowered Autonomous Institute Affiliated to University of Mumbai)
[Knowledge is Nectar]

Department of Computer Science Engineering

```
Ledger --> Tx : debitRecorded
deactivate Ledger

' 7. Create credit ledger entry
Tx -> Ledger : createCreditEntry(...)
activate Ledger

Ledger --> Tx : creditRecorded
deactivate Ledger

Tx --> Service : transactionCreated
deactivate Tx

' 8. Successful response
Service --> API : transferConfirmation()
API -> User : displayConfirmation()

else Insufficient credit

' 9. Failure response
Service --> API : transferFailed()
API -> User : displayError()

end

deactivate Service
deactivate API

@enduml
```



**BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY**
(Empowered Autonomous Institute Affiliated to University of Mumbai)
[Knowledge is Nectar]

Department of Computer Science Engineering

Output	Green Finance - Carbon Credit Transfer Sequence Diagram <pre>sequenceDiagram actor User participant GFAPI as Green Finance API participant CFS as Carbon Finance Service participant CC as CarbonCredit participant T as Transaction participant LE as LedgerEntry User->>GFAPI: transferCredit(creditId, receiverId, amount) activate GFAPI GFAPI->>CFS: processTransfer(userId, creditId, receiverId, amount) activate CFS CFS->>CC: getAvailableAmount() activate CC CC-->>CFS: availableAmount deactivate CC alt [Sufficient credit] CFS->>T: transfer(receiverId, amount) activate T T->>LE: createDebitEntry(...) activate LE LE-->>T: debitRecorded deactivate LE T->>LE: createCreditEntry(...) activate LE LE-->>T: creditRecorded deactivate LE T-->>CFS: transactionCreated deactivate T else [Insufficient credit] CFS-->>GFAPI: transferFailed() deactivate CFS end GFAPI-->>User: displayConfirmation() deactivate GFAPI GFAPI-->>User: displayError()</pre> The diagram illustrates the sequence of operations for a carbon credit transfer. It involves a User, Green Finance API, Carbon Finance Service, CarbonCredit, Transaction, and LedgerEntry. The process starts with a User initiating a transfer via the Green Finance API. The API then calls the Carbon Finance Service to process the transfer. The service checks the available amount in the CarbonCredit object. If the credit is sufficient, it proceeds to create a Transaction, which involves creating debit and credit entries in the LedgerEntry. If the credit is insufficient, the transfer fails. The process concludes with a confirmation or error message being sent back to the User via the Green Finance API.
Conclusion	Through this experiment, I learned how to design a UML Interaction/Sequence Diagram by identifying the actor, participating objects, lifelines, messages, activation bars, and conditional interactions involved in a software scenario. I understood how messages are represented in chronological order and how synchronous calls, return messages, and an <code>alt</code> fragment can be used to model different outcomes. The experiment also helped me understand the dynamic interaction between the Green Finance system components during a carbon-credit transfer process.
References	https://moodle.spit.ac.in/pluginfile.php/136644/mod_resource/content/1/26-27_Sem-V_SE_Lab_04.pdf